

```

package main

import (
    "fmt"

    "github.com/connerohnesorge/goffice/drawingml"
    "github.com/connerohnesorge/goffice/presentation"
)

// createPieChartSlide creates a slide with a pie chart showing market share.
// The chart displays market share distribution across five companies.
func createPieChartSlide(
    doc *presentation.Document,
) error {
    slidePart, err := doc.AddSlide()
    if err != nil {
        return fmt.Errorf(
            errFailedToAddSlide,
            err,
        )
    }
    slide := slidePart.Slide()

    // Add slide title
    addTitleToSlide(
        slide,
        "Market Share Distribution",
    )

    // Create chart with title
    chartSpace, chart := createChartWithTitle(
        "2024 Market Share by Company",
    )
    plotArea := chart.PlotArea()

    // Add pie chart
    pieChart := plotArea.AddPieChart()

    // Add single series with category labels and values with cached data
    series := pieChart.AddSeries(0, 0)

    // Create category axis data with cached values
    cat := drawingml.NewCategoryAxisData()
    strRef := drawingml.NewStringReferenceWithCache(
        pieCategoryRange,
        pieCategories,
    )
    cat.AppendChild(strRef)
    series.SetCategoryAxisData(cat)

    // Create values with cached numbers
    vals := drawingml.NewValues()
    numRef := drawingml.NewNumberReferenceWithCache(
        pieValuesRange,
        pieValues,
    )
    vals.AppendChild(numRef)
    series.SetValues(vals)
}

```

```

// Add shape properties with fill colors for each data point
// Pie charts need individual data point formatting
for i := range pieCategories {
    dPt := drawingml.NewDataPoint(uint32(i))
    spPr := drawingml.NewChartShapeProperties()
    colorIndex := i % len(chartColors)
    spPr.SetSolidFill(chartColors[colorIndex])
    dPt.SetShapeProperties(spPr)
    series.AppendChild(dPt)
}

// Add legend on the right side
legend := drawingml.NewLegendWithPosition(
    drawingml.LegendPositionRight,
)
chart.SetLegend(legend)

return addChartToSlide(slidePart, chartSpace)
}

// createDoughnutChartSlide creates a slide with a doughnut chart showing budget
allocation.
// The chart displays annual budget distribution across six departments.
func createDoughnutChartSlide(
    doc *presentation.Document,
) error {
    slidePart, err := doc.AddSlide()
    if err != nil {
        return fmt.Errorf(
            errFailedToAddSlide,
            err,
        )
    }
    slide := slidePart.Slide()

    // Add slide title
    addTitleToSlide(
        slide,
        "Annual Budget Allocation",
    )

    // Create chart with title
    chartSpace, chart := createChartWithTitle(
        "2024 Department Budget Distribution",
    )
    plotArea := chart.PlotArea()

    // Add doughnut chart with 50% hole size
    doughnutChart := plotArea.AddDoughnutChart()
    doughnutChart.SetHoleSize(doughnutHoleSize)

    // Add single series with category labels and values with cached data
    series := doughnutChart.AddSeries(0, 0)

    // Create category axis data with cached values
    cat := drawingml.NewCategoryAxisData()
    strRef := drawingml.NewStringReferenceWithCache(
        doughnutCategoryRange,
        doughnutCategories,
    )

```

```

)
cat.AppendChild(strRef)
series.SetCategoryAxisData(cat)

// Create values with cached numbers
vals := drawingml.NewValues()
numRef := drawingml.NewNumberReferenceWithCache(
    doughnutValuesRange,
    doughnutValues,
)
vals.AppendChild(numRef)
series.SetValues(vals)

// Add colored data points to the series
addDoughnutDataPointColors(
    series,
    doughnutCategories,
)

// Add legend on the right side
legend := drawingml.NewLegendWithPosition(
    drawingml.LegendPositionRight,
)
chart.SetLegend(legend)

return addChartToSlide(slidePart, chartSpace)
}

```